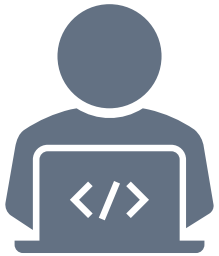
Java OOP

Object-Oriented Programming

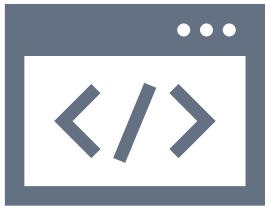
Presented By: Ahmed Hegazy Mohamed

Introduction

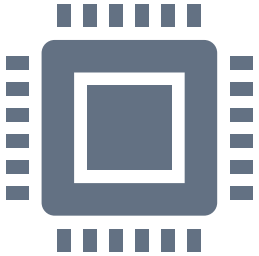
What is Java OOP?



OOP (Object-Oriented Programming) is a programming paradigm that organizes software design around **objects** rather than functions or logic.

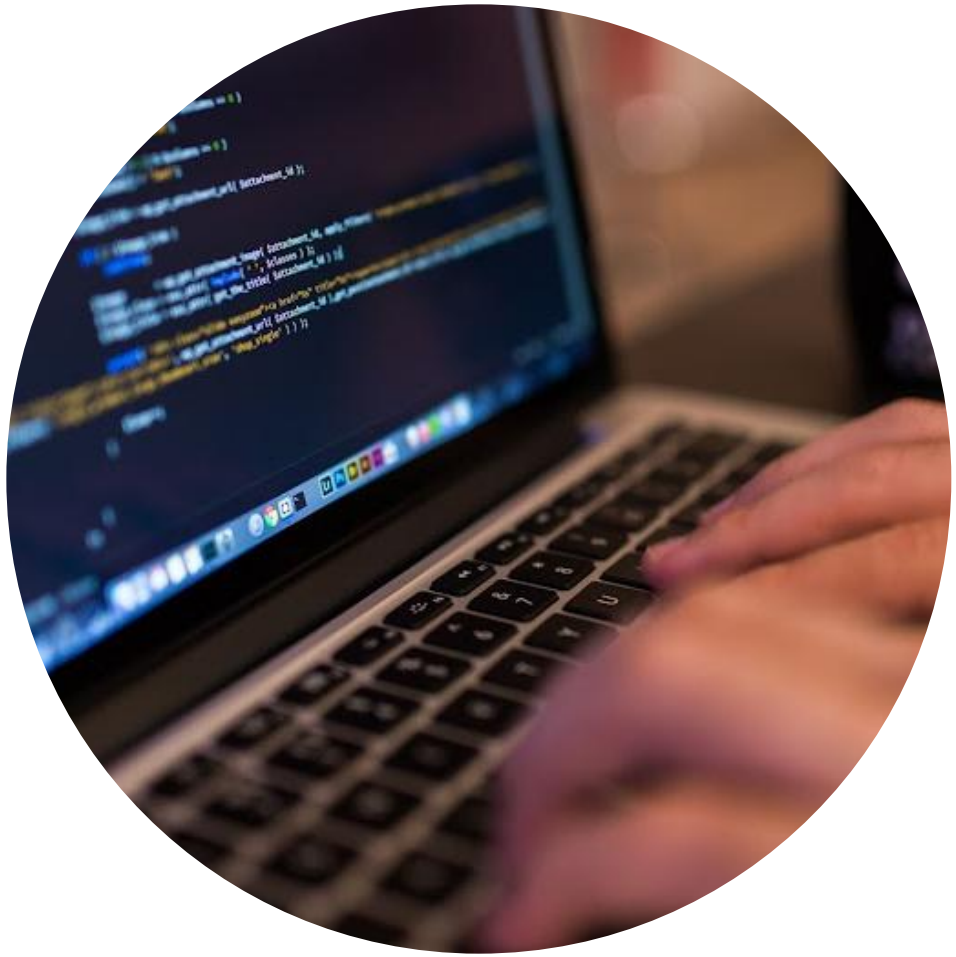


In **Java**, OOP is the core concept, which allows developers to model real-world entities and their interactions in a program.



Java OOP focuses on **encapsulation, inheritance, polymorphism, and abstraction**, making code **reusable, modular, and easier to maintain**.

Key Concepts of Java OOP



Key Concepts of Java OOP

- Classes
- Objects
- Access Modifiers
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction
- Interfaces
- Final Keyword

Classes
Objects
Access Modifiers

Classes in Java?

Is a user-defined type:

Instances of a class are objects

Describes the data (attributes)

Defines the behavior (methods)

Declaration of class

```
Public class Car {
    String color;
    int speed;

    void drive() {
      System.out.println("The car is driving at " + speed + " km/h");
    }
}
```

Objects in java

An **instance** of a class.

Each object has its own **state** (values of fields) and **behavior** (methods).

Declaration of object

```
Car myCar = new Car();
myCar.color = "Red";
myCar.speed = 100;
myCar.drive();
```

Access Modifiers in java

Access Modifiers in Java define the **visibility and accessibility** of **classes, methods, and variables**. They control **which parts of your program can use certain code**.

Modifier	Class	Package	Subclass	World (anywhere)
private	✅	❌	❌	❌
default*	✅	✅	❌	❌
protected	✅	✅	✅	❌
public	✅	✅	✅	✅

Encapsulation

Encapsulation in java

Hides **internal details** of an object and provides **access via methods.**

Achieved using **private fields** and **public getters/setters.**

Encapsulation in java

```
class Person {
    private String name;
    public void setName(String n) {
        name = n;
    }
    public String getName() {
        return name;
    }
}
Person p = new Person();
p.setName("Ahmed");
System.out.println(p.getName()); // Ahmed
```

Inheritance

Inheritance in java

Allows a class (child) to **inherit properties and methods** from another class (parent).

Promotes **code reusability**.

Inheritance in java

```
class Animal {
    void eat() {
        System.out.println("Animal eats food");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
Dog d = new Dog();
d.eat();  // inherited
d.bark(); // own method
```

Polymorphism

Polymorphism in java

One interface, multiple behaviors.

Two types:

- **Compile-time (method overloading)** – same method name, different parameters.
- **Runtime (method overriding)** – subclass provides its own implementation of a parent method.

Polymorphism in java

```
class Animal {
    void sound() {
        System.out.println("Some sound");
    }
}
class Cat extends Animal {
    void sound() {
        System.out.println("Meow");
    }
}
Animal a = new Cat();
a.sound(); // Meow (runtime polymorphism)
```

Abstraction

Abstraction in java

Hides **implementation details**, showing only **essential features**.

Achieved using **abstract classes** and **interfaces**.

Abstraction in java

```
abstract class Shape {
    abstract void area();
}
class Circle extends Shape {
    int radius;
    Circle(int r) { radius = r; }
    void area() {
        System.out.println("Area: " + (3.14 * radius * radius));
    }
}
Circle c = new Circle(5);
c.area(); // Area: 78.5
```

Interfaces

Interfaces in java

An **interface** is a **blueprint for a class.**

It defines **methods that a class must implement,**

but **does not provide implementation** (except default/static methods in Java 8+).

Used to achieve **100% abstraction** and **multiple inheritance** in Java

Interfaces in java

```
interface Vehicle {
    void start(); // abstract method
    void stop();
}
```

Implementing an Interface

```
class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts");
    }
    public void stop() {
        System.out.println("Car stops");
    }
}
public class Test {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.start(); // Car starts
        myCar.stop();  // Car stops
    }
}
```

Key Points about Interfaces

01

Supports **multiple inheritance**

02

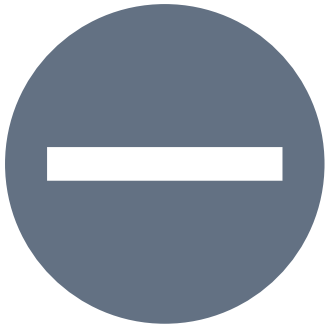
Can have **default and static methods** (Java 8+).

03

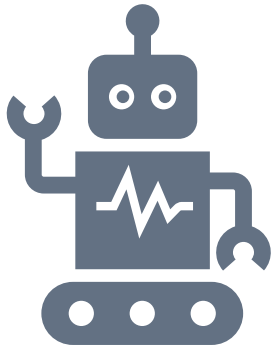
Cannot be **instantiated** directly

Final keyword

Final keyword in java



final is a **non-access modifier**.



Used to **restrict modification** of **variables, methods, and classes**.

Final variable

Value **cannot be changed** once initialized.

Example:

- final int MAX = 100;
  MAX = 200; // ❌ Error

Final method

Cannot be overridden by subclasses

Example:

```
class Parent {
 final void showMessage() {
 System.out.println("Hello");
 }
}
class Child extends Parent {
 // void showMessage() {} // ❌ Error
}
```

Final class

Cannot be subclassed.

Example:

```
final class Utils {
 static void printHello() {
 System.out.println("Hello");
 }
}
// class MyUtils extends Utils {} // ❌ Error
```

Thank you

Ahmed hegazy mohamed